Title: Debugging Closest Pair Code

CSA0613-Design and Analysis of Algorithms

Course Outcome: CO2_AT2

Student Name: A. Bhavya(192424417)

1. Problem Statement

The given brute-force Closest Pair algorithm contains logical errors in distance initialization and comparison. The task is to identify the errors, debug the code, optimize the implementation, analyze the time complexity, and rewrite the final code with proper comments.

Given Code:

```
import math

def closest_pair(points):
    min_dist = 0
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            dist = math.sqrt(
                (points[i][0] - points[j][0]) ** 2 +
                (points[i][1] - points[j][1]) ** 2
            )
            if dist > min_dist:
                min_dist = dist
    return min_dist
```

Errors Identified and Corrected Errors

Error 1: Incorrect Initialization of Minimum Distance

Original Code:

```
min_dist = 0
```

Issue:

The minimum distance is initialized to 0, which is incorrect because the distance between two different points is always greater than or equal to zero. As a result, the algorithm cannot correctly determine the smallest distance.

Corrected Code:

```
min_dist = float('inf')
```

Explanation:

The value float('inf') represents infinity. Initializing the minimum distance to infinity ensures that the first calculated distance will always be smaller and can be stored correctly.

Error 2: Incorrect Comparison Operator

Original Code:

```
if dist > min_dist:
    min_dist = dist
```

Issue:

The condition uses the greater-than operator (>), which finds the maximum distance instead of the minimum distance.

Corrected Code:

```
if dist < min_dist:
    min_dist = dist
```

Explanation:

The less-than operator (<) is used to update the minimum distance whenever a smaller distance is found.

Error 3: Missing Input Validation

Issue:

The code does not check whether the input contains at least two points. If the list contains fewer than two points, the algorithm cannot calculate the distance.

Corrected Code:

```
if len(points) < 2:
    return None
```

Explanation:

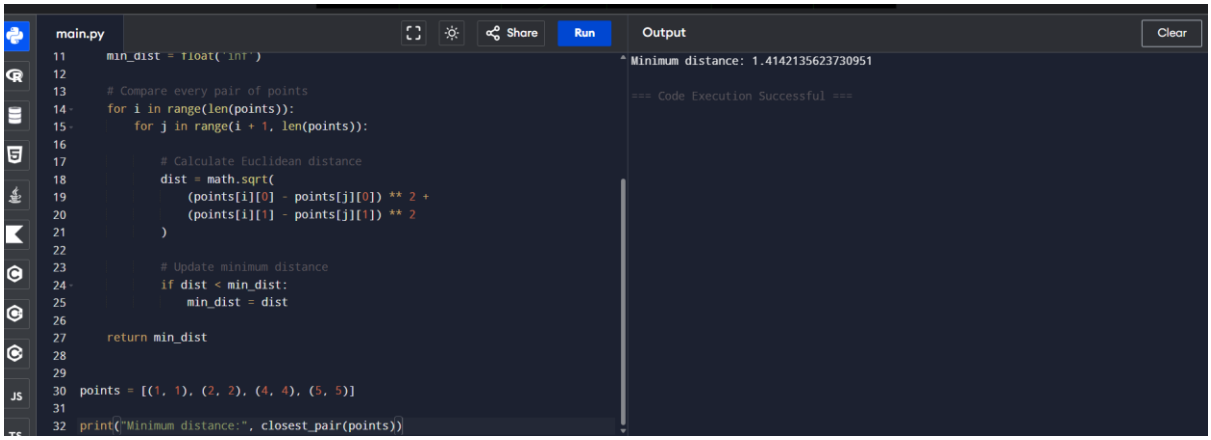
This condition ensures that the function executes only when there are at least two points available.

CORRECTED SOURCE CODE:

main.py

```python
import math
def closest_pair(points):


    if len(points) < 2:
        return None


    min_dist = float('inf')


    for i in range(len(points)):
        for j in range(i + 1, len(points)):


            dist = math.sqrt(
                (points[i][0] - points[j][0]) ** 2 +
                (points[i][1] - points[j][1]) ** 2
            )

                    min_dist = dist

```

OUTPUT:



Time Complexity :O(n)

Space Complexity :O(1)

Conclusion:

The corrected brute-force Closest Pair algorithm compares every pair of points and therefore has a time complexity of **O(n²)** and a space complexity of **O(1)**.